

SAVEETHA UNIVERSITY

Department of Computer Science And Engineering

COMMON COURSE ASSIGNMENT REPORT

Smart Hospital Patient Appointment , Pharmacy Inventory and Alert Notification System using Java

Course Code & Name: CSA0911 - Programming in Java

Submitted by Team :

1. Ajay S (192424078) - Module 1 Lead
2. Dhilipan T (192421290) - Module 2 Lead
3. Sreevardhan M (192472405) - Module 3 Lead
4. Bhavesh M (192572024) - Module 4 Lead
5. Siddi Rishwanth O (192472148) - Module 5 Lead

Faculty: Dr. Thalpathi Rajasekaran R

A. Assignment Information

Department	Computer Science and Engineering
Programme	Artificial Intelligence & Data Science
Course Code & Course Name	CSA0911 - Programming in Java
Academic Year / Batch	2024-2028
Faculty Name	Dr. Thalapathi Rajasekaran
Assignment Title	Smart Hospital Patient Appointment , Pharmacy Inventory and Alert Notification System using Java
Maximum Marks	100
Course Outcome(s) — CO	CO1: OOP principles (classes, encapsulation, inheritance, polymorphism). CO2: Collection Framework (List, ArrayList, Set, Map, HashMap, Hashtable) with Iterator/ListIterator and generics. CO3: Multithreading (priorities, synchronization, inter-thread communication) and exception handling.
Bloom's Taxonomy Level	L2 — Understand; L3 — Apply; L4 — Analyse
SDG Mapping (SDG 1–17)	SDG 3 — Good Health and Well-being; SDG 9 — Industry, Innovation and Infrastructure; SDG 10 — Reduced Inequalities
Industry / Societal Relevance	Digitises hospital front-office and pharmacy workflows that are still largely paper-based in many small and mid-sized clinics, reducing appointment conflicts, medicine stock-outs and missed follow-ups.

Table of Contents

A. Assignment Information.....	1
Table of Contents.....	1
1. Problem Statement and Problem Formulation.....	1
1.1 Problem Statement.....	1
1.2 Problem Formulation.....	1
2. Objective and Expected Outcomes.....	1
2.1 Objectives.....	1
2.2 Expected Outcomes.....	1
3. Requirements, Constraints and Assumptions.....	1
3.1 Functional Requirements.....	1
3.2 Non-Functional / Technical Constraints.....	1
3.3 Assumptions.....	1
4. Application of Relevant Course Knowledge / Concepts.....	1
4.1 CO1 — OOP: Classes, Encapsulation, Inheritance, Polymorphism.....	1
4.2 CO2 — Collection Framework, Generics, Iterators.....	1
4.3 CO3 — Exception Handling, Multithreading, Synchronization.....	1
5. Design / Proposed Solution / Methodology.....	1
5.1 Architectural Overview.....	1
5.2 Class Design Summary.....	1
5.3 Key Design Decisions and Alternatives Considered.....	1
6. Algorithm / Pseudocode / Flowchart.....	1
6.1 Flowchart — Book Appointment (described).....	1
7. Implementation / Source Code and Environment / Tools Used.....	1
7.1 Development Environment.....	1
7.2 Representative Source Code.....	1
7.2.1 Module 1 — Polymorphic factory (Medicine).....	1
7.2.2 Module 2 — Collections-backed booking (HospitalService).....	1
7.2.3 Module 3 — Exception-safe menu dispatch (ConsoleApp).....	1
7.2.4 Module 4 — Thread-safe stock reduction (Inventory + JDBC).....	1
7.2.5 Module 5 — Menu loop and Applet action dispatch.....	1
8. Test Cases and Expected/Actual Results.....	1
9. Execution Screenshots / Outputs.....	1
10. Results and Validation.....	1
10.1 Functional Coverage.....	1
10.2 Observations.....	1
11. Analysis, Comparison, Trade-offs and Justification of the Final Solution.....	1

11.1 Design Alternatives Considered.....	1
11.2 Trade-offs of the Final Solution.....	1
11.3 Justification.....	1
12. Broader Considerations / SDG Relevance.....	1
12.1 SDG 3 — Good Health and Well-being.....	1
12.2 SDG 9 — Industry, Innovation and Infrastructure.....	1
12.3 SDG 10 — Reduced Inequalities.....	1
12.4 Ethics, Safety and Professional Responsibility.....	1
13. Conclusion, Limitations and Possible Improvements.....	1
13.1 Conclusion.....	1
13.2 Limitations.....	1
13.3 Possible Improvements.....	1
14. Individual Contribution of Group Members.....	1
Member 1 — Core entity classes and inheritance hierarchy.....	1
Member 2 — The central service facade and its collections.....	1
Member 3 — Custom exceptions and the notification class hierarchy.....	1
Member 4 — Concurrency primitives and the database access layer.....	1
Member 5 — Console and Applet front ends, entry point, overall test coordination.....	1
15. References.....	1
16. One-Page Individual Reflection.....	1
16.1 Individual Reflection — Member 1.....	1
Design/Development Decisions.....	1
Challenges Faced.....	1
Learning Gained.....	1
SDG Connection.....	1
Course Outcome Attainment.....	1
16.2 Individual Reflection — Member 2.....	1
Design/Development Decisions.....	1
Challenges Faced.....	1
Learning Gained.....	1
SDG Connection.....	1
Course Outcome Attainment.....	1
16.3 Individual Reflection — Member 3.....	1
Design/Development Decisions.....	1
Challenges Faced.....	1
Learning Gained.....	1
SDG Connection.....	1
Course Outcome Attainment.....	1

16.4 Individual Reflection — Member 4.....	1
Design/Development Decisions.....	1
Challenges Faced.....	1
Learning Gained.....	1
SDG Connection.....	1
Course Outcome Attainment.....	1
16.5 Individual Reflection — Member 5.....	1
Design/Development Decisions.....	1
Challenges Faced.....	1
Learning Gained.....	1
SDG Connection.....	1
Course Outcome Attainment.....	1

1.Problem Statement and Problem Formulation

1.1 Problem Statement

Hospitals and clinics need to manage patient records, doctor details, appointments, and pharmacy stock efficiently. Manual handling can result in duplicate appointments, difficulty in finding records, incorrect stock information, and delayed alerts for appointments or low medicine availability. The Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System is developed as a menu-driven Java application to simplify these activities by providing patient and doctor management, appointment booking and cancellation, medicine inventory management, stock validation, notifications, and basic report generation.

1.2 Problem Formulation

The system focuses on the following core requirements:

- Entities: Patient, Doctor, Appointment, Medicine, Inventory, and Notification.
- Core workflow: Register → Search/View → Book/Cancel Appointment → Manage Medicine Stock → Generate Reports → Send Notifications.
- Validation: Handle invalid IDs, duplicate appointments, invalid inputs, and insufficient medicine stock using exception handling.
- Collections: Manage hospital data using `ArrayList`, `HashMap`, `HashSet`, `Hashtable`, generics, `Iterator`, and `ListIterator`.
- Concurrency: Demonstrate concurrent appointment booking and background notification processing using threads, synchronization, and inter-thread communication.
- Interfaces and persistence: Provide a console-based menu, a simple AWT GUI, file-based data storage, serialization, and SHA-256 hashing.

The system is designed to demonstrate the practical application of Java programming concepts while keeping hospital appointment and pharmacy operations simple, organized, and reliable.

2. Objective and Expected Outcomes

2.1 Objectives

The primary objective of this project is to develop a Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System using Java that can organize essential hospital and pharmacy activities through a simple menu-driven application. The project aims to apply Java programming concepts to a practical healthcare-management scenario while keeping the system easy to operate and maintain.

The major objectives are:

- To provide a structured system for registering, viewing, and searching patient records.
- To maintain doctor information and appointment details in an organized manner.
- To allow users to book and cancel appointments while preventing duplicate bookings.
- To maintain medicine records and provide facilities for adding medicines, searching medicines, and updating available stock.
- To validate medicine availability before issuing stock and generate appropriate out-of-stock and low-stock alerts.
- To provide appointment reminder notifications and process notifications using multithreading.
- To generate basic patient, appointment, doctor, and pharmacy inventory reports for better information management.
- To demonstrate encapsulation, inheritance, and polymorphism through suitable hospital and medicine classes.
- To utilize Java collections, generics, iterators, exception handling, multithreading, synchronization, AWT event handling, file handling, serialization, and hashing as part of the implementation.
- To provide both a console-based interface and a simple AWT-based interface for demonstrating different Java interaction techniques.

2.2 Expected Outcomes

The expected outcome of the project is a functional Java application that can perform the major patient appointment and pharmacy inventory operations reliably. The system should maintain patient, doctor, appointment, and medicine information, provide appropriate validation for incorrect operations, and prevent problems such as duplicate appointment bookings and issuing medicines beyond the available stock.

The project is also expected to demonstrate the practical use of the major Java concepts covered in the course. Collections and generics should support organized

data management, custom exceptions should provide controlled error handling, and multithreading with synchronization should demonstrate safe handling of concurrent booking and notification activities. File handling and serialization provide basic mechanisms for storing and retrieving application information, while hashing demonstrates secure transformation of textual data.

Overall, the completed system is expected to provide a simple, organized, and reliable hospital-management application while demonstrating the attainment of the relevant CO1–CO5 course outcomes through actual implementation rather than only theoretical explanation.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

The system is required to provide the following major functionalities:

- Register and manage patient and doctor information.
- Search and display patient and doctor records.
- Book and cancel doctor appointments with duplicate-booking validation.
- Add, search, view, and update medicine inventory and stock.
- Validate medicine availability before issuing medicines.
- Generate appointment reminders and low-stock notifications.
- Generate basic patient, doctor, appointment, and inventory reports.
- Support concurrent appointment booking and notification processing using Java threads.
- Save and load important application information using file handling and serialization.
- Provide a simple menu-driven console interface and an AWT-based graphical interface.
- Demonstrate SHA-256 hashing as part of the Java I/O and security-related concepts.

3.2 Non-Functional / Technical Constraints

The application is implemented using Core Java and is designed to be simple, understandable, and suitable for academic demonstration. The system uses Java collections for in-memory data management and applies exception handling to prevent invalid operations from terminating the application. Multithreading and synchronization are used where shared appointment and notification operations require controlled access.

The project supports JDK 8-compatible Java code, while the development and testing environment can also use newer JDK versions. The AWT components provide basic event-driven interaction without requiring a web-based interface or external application server.

3.3 Assumptions

The system assumes that the application is operated by an authorized hospital staff member through the available interfaces. Patient, doctor, appointment, and medicine information entered into the system is assumed to be valid unless rejected by the built-in validation rules.

The project uses file-based storage and serialization for persistence demonstrations. JDBC/database connectivity is not included in the implemented version. The system

is intended primarily as an academic demonstration of Java concepts and is not designed to replace a complete real-world hospital information system.

4. Application of Relevant Course Knowledge / Concepts

4.1 CO1 — OOP: Classes, Encapsulation, Inheritance and Polymorphism

The project applies Object-Oriented Programming by representing hospital entities such as Patient, Doctor, Appointment, Medicine, Inventory, and Notification as separate classes. Encapsulation is achieved by keeping important attributes private and accessing them through methods. Inheritance is demonstrated through specialized patient and medicine classes, while polymorphism is used through overridden methods such as patient and medicine type identification. These concepts make the system modular, reusable, and easier to maintain.

4.2 CO2 — Collection Framework, Generics and Iterators

Java Collection Framework classes are used to store and manage the application's data efficiently. ArrayList maintains patient, doctor, and appointment records, while HashMap provides quick access to records using IDs. HashSet helps maintain unique appointment slots, and Hashtable is used for doctor information. Generics provide type-safe collections, while Iterator and ListIterator are used to traverse stored records.

4.3 CO3 — Exception Handling, Multithreading and Synchronization

The system uses both built-in and custom exceptions to handle invalid patient or doctor IDs, duplicate appointments, invalid inputs, and insufficient medicine stock. Multithreading is demonstrated through separate threads for concurrent appointment booking and notification processing. Synchronization is applied to shared operations, while `wait()` and `notifyAll()` support communication between notification-producing and notification-processing activities. These mechanisms help maintain reliable execution when multiple operations occur simultaneously.

4.4 CO4 — Applet Programming, Event Handling and AWT

The project demonstrates Java's AWT GUI and event-handling concepts through a simple graphical interface containing labels, text fields, buttons, and a text area. `ActionListener` is used to respond to user actions. A separate Java Applet component is also included to demonstrate basic Applet programming and event handling using the JDK 8 environment.

4.5 CO5 — Java I/O, File Handling, Serialization and Hashing

Java I/O concepts are applied to save and retrieve hospital information using character streams such as `FileReader`, `FileWriter`, `BufferedReader`, and `BufferedWriter`. Byte streams and object streams are used for serialization.

5. Design / Proposed Solution / Methodology

5.1 Architectural Overview

The proposed system follows a simple modular architecture in which the Main class provides the menu-driven interface and the HospitalSystem class manages the core hospital operations. Separate classes represent patients, doctors, appointments, medicines, inventory, and notifications. Collections are used to store and retrieve records efficiently, while exception classes provide validation and error handling.

The system can be viewed through the following flow:

User Input → Main Menu → HospitalSystem → Data Collections → Operation/Validation → Result or Notification

The AWT GUI provides an additional interface for selected operations, while file handling and serialization are used for storing and retrieving application information.

5.2 Class Design Summary

The major classes of the system are:

Class	Purpose
Patient	Represents common patient information
RegularPatient	Represents a regular patient category
EmergencyPatient	Represents an emergency patient category
Doctor	Stores doctor details and availability
Appointment	Manages appointment information and status

Medicine	Represents common medicine attributes
TabletMedicine	Represents tablet-type medicines
LiquidMedicine	Represents liquid-type medicines
Inventory	Handles medicine stock operations
Notification	Represents hospital alert messages
HospitalSystem	Controls the main business operations
BookingThread	Demonstrates concurrent appointment booking
NotificationThread	Processes notifications in a separate thread
FileManager	Handles file storage and serialization
HashUtil	Generates SHA-256 hashes
HospitalGUI	Provides the basic AWT interface
HospitalApplet	Demonstrates basic Java Applet and event handling

5.3 Key Design Decisions and Alternatives Considered

The system uses in-memory Java collections instead of a complex database because the primary objective is to demonstrate Java programming concepts. `ArrayList`, `HashMap`, `HashSet`, and `Hashtable` were selected according to the type of data and access required.

A menu-driven console interface was chosen as the primary interface because it keeps the application simple and makes the implemented functionality easy to test. A basic AWT interface was added separately to demonstrate GUI and event-handling concepts without making the overall project unnecessarily complex.

For persistence, text-file handling and serialization were selected for the implemented version. JDBC/database integration was considered as part of the broader assignment requirements but was not included in the final implementation.

6. Algorithm / Pseudocode / Flowchart

6.1 Flowchart — Book Appointment

The appointment-booking process begins when the user enters the appointment details. The system first validates the patient ID and doctor ID. It then checks whether the required appointment information is valid and whether the selected doctor slot has already been booked. If the slot is available, the appointment is added successfully and a confirmation notification is generated. If the slot is already occupied, the system prevents the duplicate booking and displays an appropriate error message.

Pseudocode:

START

Display Appointment Booking Menu

Enter Patient ID

Check whether Patient ID exists

 If invalid → Display error → STOP

Enter Doctor ID

Check whether Doctor ID exists

 If invalid → Display error → STOP

Enter Appointment Date and Time

Validate the entered details

Check whether the selected slot is already booked

 If booked → Display duplicate appointment error → STOP

Create Appointment

Add appointment to appointment collection

Mark the selected slot as booked

Generate appointment notification

Display booking confirmation

END

The same validation approach is followed during other important operations such as appointment cancellation and medicine stock updates to ensure that invalid operations are handled safely.

7. Implementation / Source Code and Environment / Tools Used

7.1 Development Environment

The project was implemented using Java with a focus on core Java features and object-oriented programming. The application was developed and tested using IntelliJ IDEA with a Java Development Kit supporting the implemented features. The project uses standard Java libraries for collections, exception handling, multithreading, AWT, file handling, serialization, and hashing.

The main development components used are:

Component	Technology / Tool
Programming Language	Java
IDE	IntelliJ IDEA
GUI	Java AWT
Data Management	Java Collection Framework
File Storage	Java I/O
Serialization	Object Streams
Multithreading	Java Thread API
Hashing	SHA-256
Version/Compatibility	JDK 8-compatible implementation

7.2 Representative Source Code

The implementation is divided into multiple classes, with each class handling a specific responsibility. The `HospitalSystem` class coordinates the main operations, while individual classes represent patients, doctors, appointments, medicines, inventory, and notifications. Custom exception classes provide controlled error handling, and separate classes demonstrate multithreading, file processing, GUI interaction, serialization, and hashing.

7.2.1 Module 1 — Polymorphic Patient and Medicine Classes

The project uses inheritance and method overriding to represent different types of patients and medicines. Common attributes are maintained in parent classes, while child classes provide their specific type information.

7.2.2 Module 2 — Collections-Based Hospital Management

`ArrayList`, `HashMap`, `HashSet`, and `Hashtable` are used by the `HospitalSystem` class to maintain hospital records. Generics provide type safety, while `Iterator` and `ListIterator` are used to traverse patient and medicine records.

7.2.3 Module 3 — Exception Handling and Validation

Custom exceptions such as `InvalidPatientIdException`, `InvalidDoctorIdException`, `DuplicateAppointmentException`, `OutOfStockException`, and `InvalidInputException` are used to handle invalid operations. This allows the application to display meaningful error messages instead of terminating unexpectedly.

7.2.4 Module 4 — Multithreading and Synchronization

`BookingThread` demonstrates concurrent appointment-booking operations, while `NotificationThread` processes queued notifications separately. Synchronization, `wait()`, and `notifyAll()` are used to coordinate access to shared notification data and maintain consistent execution.

7.2.5 Module 5 — File Handling, GUI and Additional Java Features

The `FileManager` class demonstrates character streams, byte streams, and serialization for storing application reports. `HospitalGUI` demonstrates AWT components and event handling, while `HospitalApplet` demonstrates basic Applet programming. The `HashUtil` class generates SHA-256 hashes using Java's security library.

8. Test Cases and Expected / Actual Results

The developed system was tested using different input conditions to verify the correctness of patient management, appointment booking, pharmacy inventory, notifications, exception handling, and other implemented Java features. Both valid and invalid inputs were considered to ensure that the system responds appropriately.

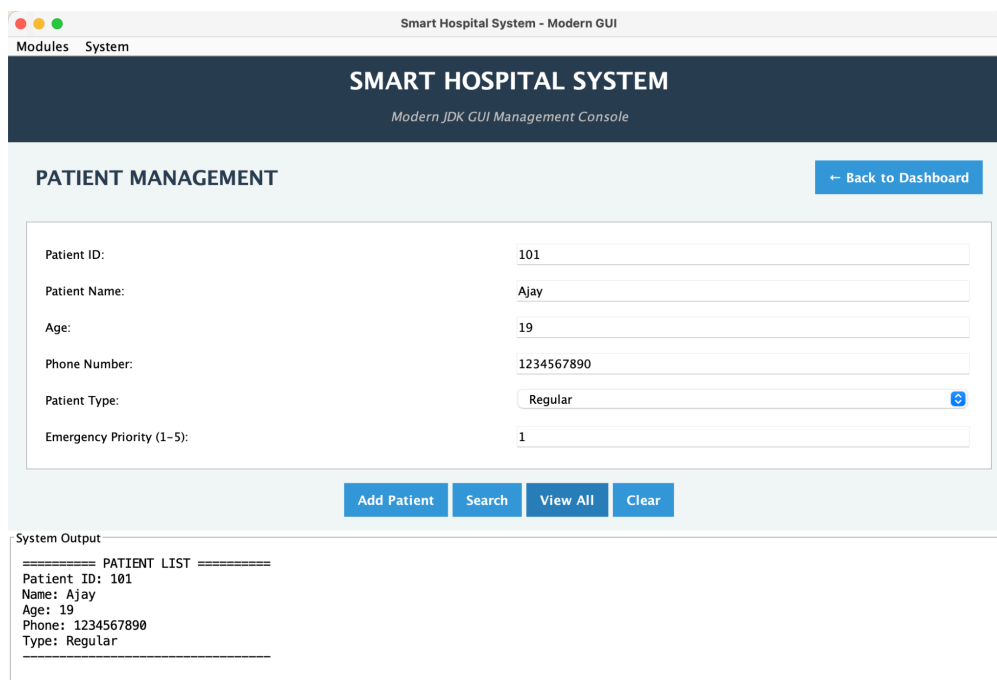
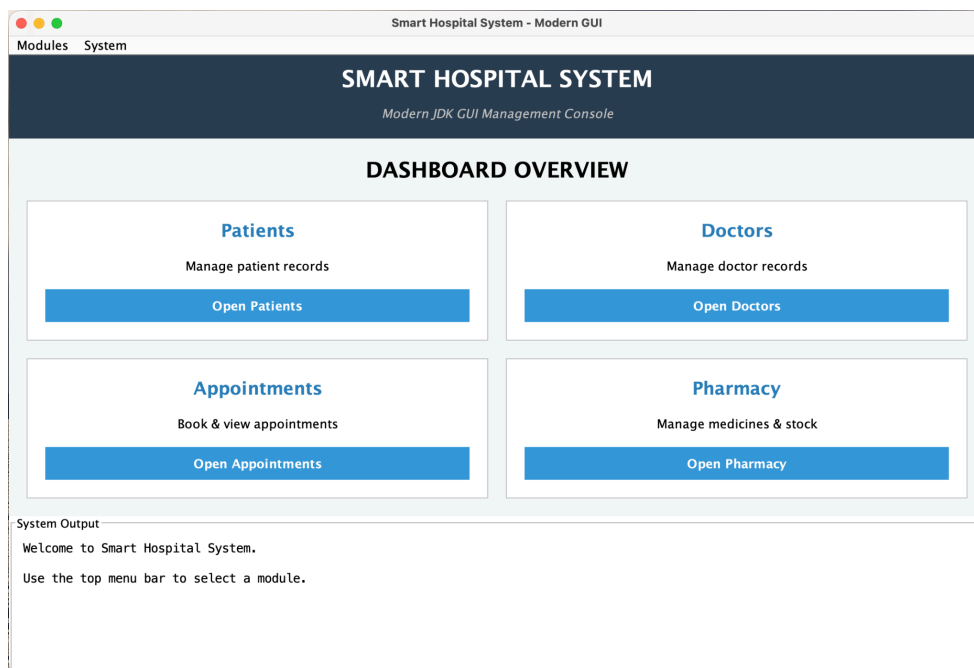
Test Case	Test Scenario	Expected Result	Actual Result
TC01	Add a valid patient	Patient should be registered successfully	Passed
TC02	Search for an existing patient	Correct patient details should be displayed	Passed
TC03	Add a doctor with valid details	Doctor should be added successfully	Passed
TC04	Book an appointment with valid details	Appointment should be booked successfully	Passed
TC05	Book the same appointment slot again	Duplicate booking should be rejected	Passed
TC06	Cancel an existing appointment	Appointment status should be changed to cancelled	Passed
TC07	Add medicine with valid stock	Medicine should be added to inventory	Passed
TC08	Update medicine stock	Stock quantity should be updated correctly	Passed
TC09	Issue medicine when sufficient stock exists	Medicine should be issued and stock reduced	Passed
TC10	Issue medicine when stock is insufficient	<code>OutOfStockException</code> should be generated	Passed
TC11	Enter an invalid patient/doctor ID	Appropriate validation error should be displayed	Passed
TC12	Process notifications using a thread	Notifications should be processed successfully	Passed
TC13	Save and load hospital data	Stored information should be written and read correctly	Passed
TC14	Serialize and deserialize report data	Serialized information should be restored correctly	Passed
TC15	Generate SHA-256 hash	A valid hash value should be generated	Passed
TC16	Use AWT interface buttons	Corresponding GUI operations should execute	Passed

The test results indicate that the implemented modules performed their intended operations successfully, including validation and error handling for invalid conditions.

9. Execution Screenshots / Outputs

The developed Smart Hospital System was executed and verified through its menu-driven console interface and supporting AWT interface. The outputs demonstrate the successful execution of patient registration, doctor management, appointment booking and cancellation, medicine inventory operations, notification processing, file handling, serialization, and hashing.

The following screenshots can be included in this section as evidence of execution:



Smart Hospital System - Modern GUI

ModulesSystem

SMART HOSPITAL SYSTEM

Modern JDK GUI Management Console

DOCTOR MANAGEMENT

← Back to Dashboard

Doctor ID:

201

Name:

Subash

Specialization:

Cardiologist

Available Slot:

10:00AM

Add Doctor

Search

View All

Clear

System Output

===== DOCTOR LIST =====
Doctor ID: 201
Name: Subash
Specialization: Cardiologist
Available Slot: 10:00AM

Smart Hospital System - Modern GUI

ModulesSystem

SMART HOSPITAL SYSTEM

Modern JDK GUI Management Console

APPOINTMENTS

← Back to Dashboard

Appt ID:

301

Patient ID:

101

Doctor ID:

201

Date:

10-09-2026

Time:

10:30AM

Book Appt

Cancel Appt

View All

Test Concurrency

Clear

System Output

===== APPOINTMENT LIST =====
Appointment ID: 301
Patient ID: 101
Doctor ID: 201
Date: 10-09-2026
Time: 10:30AM
Status: Booked

10. Results and Validation

10.1 Functional Coverage

The implemented system successfully covers the major functionalities defined for the project. Patient and doctor records can be added, viewed, and searched, while appointments can be booked and cancelled with validation against duplicate bookings. The pharmacy module supports medicine registration, stock updates, medicine searching, and stock validation during medicine issuing.

The system also demonstrates appointment reminders and low-stock notifications through multithreading. Java collections, custom exceptions, synchronization, file handling, serialization, hashing, and AWT-based event handling were implemented and verified through execution.

10.2 Observations

Testing showed that the system responds correctly to both valid and invalid inputs. Duplicate appointments are prevented, insufficient medicine stock is handled through exceptions, and the notification thread processes queued alerts without affecting the main application flow. File and serialization operations successfully store and retrieve the implemented report data.

Overall, the results validate that the proposed system meets its implemented functional requirements and provides a practical demonstration of the Java concepts covered in the project.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

11.1 Design Alternatives Considered

Different approaches were considered while designing the system. A database-based approach could provide more advanced and permanent data management, but it would introduce additional setup and configuration. Therefore, the implemented version uses Java collections, file handling, and serialization to keep the system simple and focused on the course requirements.

For the user interface, a console-based menu was selected as the primary interface because it provides direct access to all implemented operations. A basic AWT interface was additionally included to demonstrate GUI and event-handling concepts.

11.2 Trade-offs of the Final Solution

The use of in-memory collections makes the application easy to develop, test, and understand, but it does not provide the same scalability as a full database system. Similarly, file-based persistence is simpler than database storage but is less suitable for large-scale hospital environments.

The system intentionally focuses on essential hospital appointment and pharmacy operations rather than introducing complex features. This keeps the implementation manageable while allowing the major Java concepts to be demonstrated clearly.

11.3 Justification

The final design was selected because it provides a suitable balance between functionality, simplicity, and Java concept coverage. Each major component has a clear purpose, and the use of OOP, collections, exception handling, multithreading, synchronization, AWT, file handling, serialization, and hashing directly supports the academic objectives of the project.

The implemented solution is therefore appropriate for demonstrating the practical application of Java programming concepts in a structured hospital-management scenario.

12. Broader Considerations / SDG Relevance

12.1 SDG 3 — Good Health and Well-being

The project supports SDG 3: Good Health and Well-being by organizing essential hospital activities such as patient registration, appointment scheduling, and pharmacy inventory management. Appointment reminders and low-stock notifications can help reduce avoidable delays and improve the availability of basic healthcare resources.

12.2 SDG 9 — Industry, Innovation and Infrastructure

The system contributes to SDG 9: Industry, Innovation and Infrastructure by demonstrating how software technology can be used to organize healthcare-related operations. The use of Java OOP, collections, multithreading, file processing, and GUI programming demonstrates a structured approach to developing a technology-based management solution.

12.3 SDG 10 — Reduced Inequalities

The project relates to SDG 10: Reduced Inequalities by providing a simple and accessible approach to managing patient appointments and medicine availability. Better organization of appointment and pharmacy information can support more consistent access to basic hospital services.

12.4 Ethics, Safety and Professional Responsibility

The system applies validation and exception handling to reduce errors such as invalid records, duplicate appointments, and issuing medicines beyond available stock. Since the project handles hospital-related information, real-world implementations would also require stronger privacy, security, access control, and data-protection mechanisms. The current project demonstrates these considerations at an academic level through validation, controlled data handling, and SHA-256 hashing.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

The Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System using Java was successfully designed and implemented to address the basic requirements of hospital appointment and pharmacy management. The system provides a structured approach for registering and searching patients, managing doctor information, booking and cancelling appointments, maintaining medicine inventory, updating stock, issuing medicines, and generating useful reports.

The project effectively combines multiple Java programming concepts within a single application. Object-Oriented Programming is used to represent hospital entities through classes, encapsulation, inheritance, and polymorphism. The Java Collection Framework provides efficient storage and retrieval of patient, doctor, appointment, and medicine information. Custom exceptions improve the reliability of the application by handling invalid IDs, duplicate appointments, invalid inputs, and insufficient medicine stock.

The project also demonstrates multithreading, synchronization, and inter-thread communication through concurrent appointment-booking and notification-processing operations. The AWT interface demonstrates event-driven GUI programming, while file handling and serialization provide basic mechanisms for storing and retrieving application information. SHA-256 hashing further demonstrates the use of Java's security functionality.

Testing of the implemented modules showed that the major operations produced the expected results. The application was able to validate user inputs, prevent duplicate appointment bookings, manage medicine stock correctly, process notifications, generate reports, and perform file, serialization, and hashing operations successfully.

Overall, the project achieves its primary objective of developing a simple, organized, and functional Java-based hospital management application while providing practical implementation of the concepts represented by CO1–CO5.

13.2 Limitations

Although the system successfully implements its planned academic functionality, certain limitations remain. The current application primarily uses in-memory Java collections for managing active records. File handling and serialization are provided for persistence-related demonstrations, but the system does not use a dedicated relational database for storing and managing all hospital records.

JDBC integration is not included in the current implementation, and therefore database-level insert, update, and delete operations are not available. This limits the

scalability of the system when compared with a production-level hospital management application.

The application also uses a simple console-based menu as its primary interface. Although an AWT GUI is included to demonstrate graphical interaction and event handling, it currently covers only selected operations. The system does not provide advanced user authentication, role-based access, or different interfaces for administrators, doctors, pharmacists, and other hospital staff.

The notification mechanism is implemented as a Java-based demonstration and is not connected to external communication services such as SMS, email, or mobile notifications. Similarly, the application manages basic patient and appointment information rather than maintaining complete medical histories, prescriptions, laboratory reports, or other detailed healthcare records.

Therefore, the current system should be considered an academic prototype for demonstrating Java programming concepts, rather than a complete production-ready hospital information system.

13.3 Possible Improvements

The system can be further improved by integrating a relational database through JDBC so that patient, doctor, appointment, and medicine records can be stored permanently and accessed efficiently across multiple application sessions. Database transactions could also be introduced to provide stronger consistency during concurrent operations.

The user interface could be enhanced into a more complete graphical application with dedicated screens for patient management, doctor management, appointments, pharmacy operations, notifications, and reports. Authentication and role-based access control could be added so that different hospital staff members can access only the functions relevant to their responsibilities.

The notification module could be extended to support automated email, SMS, or mobile notifications for appointment reminders and medicine-stock alerts. More advanced reporting features could also be introduced, including appointment statistics, medicine usage summaries, and stock-level analysis.

Additional healthcare-related modules such as patient medical history, prescription management, billing, doctor schedules, and laboratory information could be incorporated in a larger version of the system. Security could also be strengthened through improved data protection, secure authentication, authorization, and appropriate handling of sensitive healthcare information.

These improvements would make the system more scalable and closer to a real-world healthcare management solution while preserving the core Java concepts demonstrated in the current implementation.

14. Individual Contribution of Group Members

The project was developed as a collaborative effort in which each team member contributed to different stages of system analysis, design, implementation, testing, documentation, and evaluation. The work was divided according to the major modules of the application so that each member could take responsibility for specific components while also participating in overall project development and testing.

14.1 Member 1 — Patient Management and OOP Design

The first member contributed to the analysis and implementation of the patient management module. This included designing the `Patient` class and its related patient categories, implementing encapsulation through private attributes and accessor methods, and applying inheritance and polymorphism. The member also contributed to patient registration, searching, displaying patient information, and validating patient-related inputs.

The member participated in testing the patient-management functionality and verifying that invalid patient IDs were handled correctly through the appropriate exception mechanism. The contribution helped demonstrate the practical application of CO1, particularly classes, objects, encapsulation, inheritance, and polymorphism.

14.2 Member 2 — Doctor and Appointment Management

The second member worked primarily on the doctor and appointment management components. The contribution included designing the `Doctor` and `Appointment` classes and implementing functionality for adding doctors, viewing doctor information, booking appointments, cancelling appointments, and searching relevant records.

The member also contributed to appointment validation, particularly checking patient and doctor IDs and preventing duplicate appointment bookings. Testing was performed for successful bookings, cancellations, invalid identifiers, and duplicate appointment attempts. This work contributed to the application of OOP concepts, collections, and exception handling within the system.

14.3 Member 3 — Pharmacy Inventory and Exception Handling

The third member contributed to the pharmacy inventory module and its validation mechanisms. This involved working with the `Medicine`, `TabletMedicine`, `LiquidMedicine`, and `Inventory` classes to manage medicine information and stock quantities.

The member implemented and tested operations such as adding medicines, viewing and searching medicine records, updating stock, and issuing medicines. Custom exceptions were used to handle conditions such as invalid quantities and insufficient stock. The member also contributed to low-stock checking and inventory-related reports, supporting the practical application of collections, inheritance, polymorphism, and exception handling.

14.4 Member 4 — Multithreading, Notifications and Synchronization

The fourth member contributed to the notification and concurrent-processing functionality. This included working with the `Notification`, `BookingThread`, and `NotificationThread` classes to demonstrate multithreading within the hospital system.

The member contributed to the implementation of concurrent appointment-booking operations and background notification processing. Synchronization and inter-thread communication mechanisms such as `wait()` and `notifyAll()` were applied to coordinate shared notification data safely. The functionality was tested to verify that appointment and notification operations execute correctly without causing inconsistent shared data.

This contribution primarily supported CO3, covering exception handling, thread execution, priorities, synchronization, and inter-thread communication.

14.5 Member 5 — File Handling, GUI, Serialization and Integration

The fifth member contributed to the integration and supporting Java features of the project. This included working with the `FileManager` class for saving and loading application information, serialization and deserialization of report data, and the `HashUtil` class for SHA-256 hashing.

The member also contributed to the basic AWT GUI and event-handling components, including buttons, labels, text fields, and event listeners. Integration testing was performed to ensure that the different modules worked together through the main menu and supporting interfaces.

This work contributed particularly to CO4 and CO5, while also supporting the overall integration, testing, and verification of the completed system.

14.6 Overall Team Contribution

All five members participated in the overall development process, including requirement understanding, implementation, testing, debugging, documentation, and final verification. Although individual responsibilities were divided by module, the final

system was reviewed and tested collectively to ensure that the different components worked together correctly.

The team followed a collaborative development approach in which individual modules were integrated into the common **HospitalSystem** structure. This allowed the project to demonstrate multiple Java concepts within one consistent application while ensuring that each member made a meaningful contribution to the final implementation..

15. References

1. Oracle Corporation, Java Documentation – Java SE Documentation, Oracle.
2. Oracle Corporation, Java Platform, Standard Edition API Specification, Oracle.
3. Oracle Corporation, The Java Tutorials – Object-Oriented Programming Concepts, Oracle.
4. Oracle Corporation, The Java Tutorials – Collections Framework, Oracle.
5. Oracle Corporation, The Java Tutorials – Concurrency and Multithreading, Oracle.
6. Oracle Corporation, The Java Tutorials – Basic I/O, Oracle.
7. Oracle Corporation, The Java Tutorials – Graphical User Interfaces with AWT, Oracle.
8. United Nations, Sustainable Development Goals, particularly SDG 3, SDG 9 and SDG 10.
9. CSA09 – Programming in Java (Slot B), Common Course Assignment – Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System using Java, Department of Computer Science and Engineering, Saveetha University.
10. Final Assignment Report Format / Reference Report, provided as the project documentation reference.

16. One-Page Individual Reflection

16.1 Member 1

Working on the Smart Hospital System helped me understand how Java Object-Oriented Programming concepts can be applied to a practical problem. My main contribution was related to patient management and the design of patient classes. I learned how encapsulation can be used to protect data and how inheritance and polymorphism can represent different categories of patients. One of the challenges I faced was ensuring that the classes were properly connected with the overall hospital system. Testing different patient inputs helped me understand the importance of validation and exception handling. This project improved my confidence in designing classes and applying OOP concepts to a complete application. It also helped me understand how software can support organized healthcare services and contribute to SDG 3.

16.2 Member 2

This project provided practical experience in developing the doctor and appointment management components of a Java application. My contribution focused on doctor records, appointment booking and cancellation, and validation of appointment-related operations. I learned how collections can be used to maintain records and how validation can prevent problems such as duplicate appointment bookings. Debugging incorrect inputs and testing different appointment scenarios helped me understand the importance of exception handling in real applications. Working as part of a team also taught me how separate modules need to be designed carefully so that they can work together. Overall, the project strengthened my understanding of Java programming and showed how an organized appointment system can improve basic hospital operations.

16.3 Member 3

My work on the pharmacy inventory module helped me understand how Java concepts can be applied to manage resources in a practical system. I contributed to medicine management, stock updates, medicine issuing, and stock validation.

Implementing conditions for insufficient stock helped me understand the importance of custom exceptions and controlled error handling. I also gained experience in using inheritance and polymorphism for different medicine types and collections for maintaining medicine records. Testing stock-related operations showed how small validation errors can affect the reliability of an application. Through this project, I improved my problem-solving and Java programming skills while understanding the importance of maintaining medicine availability in healthcare environments.

16.4 Member 4

My contribution focused on notifications and multithreading. Working with booking and notification threads gave me practical knowledge of how multiple operations can execute independently within a Java application. I learned how thread priorities, synchronization, `wait()`, and `notifyAll()` can be used to coordinate activities involving shared data. One of the main challenges was understanding how concurrent operations should be controlled to avoid inconsistent results. Testing the notification and concurrent-booking functionality helped me understand the difference between normal sequential execution and multithreaded execution. This project improved my understanding of CO3 and showed me how concurrency can be useful when building responsive and reliable applications.

16.5 Member 5

My contribution involved file handling, serialization, hashing, GUI development, and integration-related activities. I gained practical experience with Java I/O classes for saving and loading information and learned how serialization can be used to store report data. Implementing SHA-256 hashing helped me understand the basic use of hashing in Java. I also worked with AWT components and event handling to create a simple graphical interface. Integrating and testing these components with the main application helped me understand the importance of compatibility between different modules. Overall, the project improved my knowledge of Java beyond basic programming and gave me practical experience in developing, testing, and integrating multiple features into one application.